

KT24403 Operating System

Lab Project

15 Marks

DUE Date: Week 14 (27 JUNE 2024)

The Reader-Writers Problem of Threads with Locks/Semaphores

Submission: Report and Video Link Presentation through iTel.

Each group: 4 – 5 members.

The reader-writer problem is used to illustrate synchronization for threads that's are reading and writing to a shared file. Readers read files but do not update files, Writers read and update files. In the reader-writers problem, multiple readers can read at the same time but only ONE writer can be allowed to write to the file.

1. When 1 thread writes, no other threads can write.
2. When 1 thread writes, no other threads can read.
3. When 1 thread reads, no other threads can write.
4. When 1 thread reads, other threads can read ONLY.

Mutex Locks and Semaphores can be used to carry out the synchronization of readers and writers in this problem.

Create your algorithm to execute the reader-writers problem with locks to control reading and writing to ONE shared file.

There are 3 universal control variables

Mutex semaphore : ensure mutual exclusion when read_count is updated

Write_mutex: mutual exclusion lock for writers

Read_count : keep track of how many processes are currently reading object

The implementation can be written in C/C++/Java on Linux/UNIX or Windows platform.

For your program, there should be at least 4 functions, the Check function is up to you to implement whether separately or in the Read()/Write()).

1. Wait()
 - When a reader or writer needs to acquire the Mutex Semaphore or Write_mutex
 - If a reader allowed to read, Read_Count as to be incremented
2. Signal()
 - When a reader or writer finishes reading and writing, the Mutex Semaphore or Write_mutex should be released
 - If a reader finishes reading, Read_Count should be decremented
3. Read()
 - If a reader wants to read, it has to check whether any writer is writing to the file. If there are writers writing, readers are not allowed to read.
 - If there are no writing done then readers can read. Before reading, readers have to Wait() to increment Read_Count
4. Write()
 - If a writer wants to write, it has to check whether any writer is writing to the file or whether any readers are reading the file. If there are writers writing or readers reading, the writer is not allowed to write.
 - If there are no writing or reading done then writers can write. Before writing, writers have to Wait() to use the Write_mutex.
5. Check()
 - Check whether readers finish reading, Signal() to decrement Read_Count
 - Check whether writers finish writing, Signal() to release Write_mutex.

Inputs:

- Read/ Write function
- Finish/ Not Finish writing
- Finish/ Not Finish Reading

Outputs:

- Allowed/ Not Allowed to Write
- Allowed/ Not Allowed to Read
- Any Writer writing
- Number of Readers reading

The flow of the algorithm is as shown.

Choose operation Read/Write

If **Read**, Check any Writer writing,

 If **Writer writing**, check if finished writing

 If **Finish writing**, Allow to read

 Release Write_mutex

 Increment Read_Count

 If **Not Finish writing**, Not allowed to read

 If **No Writer writing**, Allow to read

If **Write**, Check any Writer writing or any Reader reading,

 If **No Writer writing and No Reader reading**, Allow to write

 Acquire Write_mutex

 If **Writer writing and No Reader reading**, check if finished writing

 If **Finish writing**, Allow to write

 Acquire Write_mutex

 If **Not Finish writing**, Not allowed to write

 If **No Writer writing but Reader reading**, check if ALL reader finish reading

 If **All Finish reading**, allowed to write

 Decrement Read_Count

 Acquire Write_mutex

 If **not All Finish reading**, Not allowed to write

 If **Writer writing and Reader reading**, // Not supposed to happen !

Your project will be evaluated based on

Program Design and Completeness (5%)

Complete Test Scenarios (7%)

Group Task Distribution and Cooperation (3%)

Some sample test scenarios.

<pre>You are currently accessing A file. Are you a Reader(r) or Writer(w)? Type e to exit. >>>r *** You are a Reader. *** Reading allowed. mutex on read count: 1 ----- running again ----- Are you a Reader(r) or Writer(w)? Type e to exit. >>>r *** You are a Reader. *** Reading allowed. mutex on read count: 2 ----- running again ----- Are you a Reader(r) or Writer(w)? Type e to exit. >>>e ----- Process exited after 16.9 seconds with return value 0 Press any key to continue . . .</pre>	<pre>You are currently accessing A file. Are you a Reader(r) or Writer(w)? Type e to exit. >>>w *** You are a Writer. *** Writing allowed. write mutex on ----- running again ----- Are you a Reader(r) or Writer(w)? Type e to exit. >>>r *** You are a Reader. *** Reading not allowed. Writer is writing. Is the Writer done writing (y/n)? y Writer done. Please proceed again ----- running again ----- Are you a Reader(r) or Writer(w)? Type e to exit. >>>r *** You are a Reader. *** Reading allowed. mutex on read count: 1 ----- running again ----- Are you a Reader(r) or Writer(w)? Type e to exit. >>>e ----- Process exited after 29.6 seconds with return value 0 Press any key to continue . . .</pre>
--	---

Your program design do not need to be the same but should have proper notification and warnings. Your output should also be able to show the state and response of the program.

Report Content to upload on iTel (Only 1 representative to submit)

- Group members and task distribution
- Code (adjust font and spacings so that looks neat and readable on report)
- Screenshots of test scenarios done

Group presentation to demo the test scenarios will be conducted on Week 14

Rubrics

Program Design and Completeness (5 Marks)

	0	1	2	
Program Design (Complete reader and writers functions – 2M) 1. Readers check condition before allow to read 2. Writers check condition before allow to read 3. Raise error if not allow to read	Missing functions or functions not properly divided	All stated functions available Functions not logically divided according to stated in question	All stated functions available Functions logically divided according to stated in question	
Complete input and output – 2 M Input: - Reader/ Writer - Previous reader/ writer status Output: - Allow to read/ write - Not allow to read/write with reason - Error message	Incomplete and directions not clear	Complete but unclear directions for users	Complete with clear directions	
Code tidiness and comments – 1 M	Not Readable and no comments	Readable with comments		

Test Scenario (7 Marks)

Test Scenario	Reader/ Writer	Marks (if no error)	
Writer writing	Reader	1	
No writer writing	Reader	1	
No Writer writing and No Reader reading	Writer	1	
Writer writing and No Reader reading	Writer	1	
No Writer writing but Reader reading	Writer	1	
Error Handling	Reader/Writer	2 if complete 1 if not complete	

Task Distribution and Cooperation (3 Marks)

Marks	1	2	3
Condition	Some members only involved to present, no participation in testing and coding	Less coders compared to testers	Fair Distribution More coders compared to testers